

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №0

«Решение уравнения 6 степени»

Выполнили:

Сивоконь Александр Александрович
студент группы N3247

(подпись)

Проверил:

Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Оглавление

Постановка задачи	2
Описание методов	2
Маршрут программы	4
Входные и выходные данные	6
Блок-схема	14
Код программы	15
Примеры работы программы	10
Заключение	24

Постановка задачи

Необходимо реализовать алгоритм, который решает уравнение шестой степени

$$ax^6 + bx^5 + cx^4 + dx^3 + kx^2 + mx + n = 0$$

используя различные численные методы в зависимости от степени уравнения.

Описание методов

Алгоритм решения:

При $a \neq 0$ (уравнение 6-й степени) применяется комбинированный метод хорд и касательных. Найденный корень используется для понижения степени многочлена по схеме Горнера. Далее:

- при степени 5 — метод хорд;
- при степени 4 — метод касательных (Ньютона);
- при степени 3 — метод Кардано;
- при степени 2 — формулы Виета;
- при степени 1 — линейное уравнение.

При $a = 0, b \neq 0$ (уравнение 5-й степени) применяется метод хорд с последующим понижением степени.

При $a = 0, b = 0, c \neq 0$ (уравнение 4-й степени) применяется метод касательных с последующим понижением степени.

При $a = 0, b = 0, c = 0, d \neq 0$ (уравнение 3-й степени) — метод Кардано.

При $a = b = c = d = k = 0, m \neq 0$ — линейное уравнение $mx + n = 0$.

Если все коэффициенты равны нулю и $n \neq 0$, уравнение не имеет решений. Если все коэффициенты, включая n , равны нулю, корнем является любое число.

Дополнительные механизмы:

Перед применением численных методов выполняется извлечение корней $x = 0$ путём снятия замыкающих нулевых коэффициентов. Для корней чётной кратности, при которых полином не меняет знак (и метод отрезков не находит смену знака), реализован поиск через корни производной: если корень $f'(x) = 0$ одновременно является корнем $f(x) = 0$, он добавляется

как корень чётной кратности.

Для поиска начального отрезка, содержащего корень, используется оценка корней по формуле Коши и сканирование с уменьшающимся шагом. Для предотвращения зависания при отсутствии действительных корней введено ограничение на общее число итераций сканирования.

Маршрут программы

1. **Ввод коэффициентов** (`input_coefficients`). Пользователь вводит коэффициенты a, b, c, d, k, m, n уравнения $ax^6 + bx^5 + cx^4 + dx^3 + kx^2 + mx + n = 0$.
2. **Определение степени уравнения** (`main`). По старшему ненулевому коэффициенту определяется степень уравнения и формируется массив коэффициентов `Mas`.
3. **Выбор метода решения** (`solve_numerical, main`):
 - степень ≥ 4 — переход к численному решателю `solve_numerical`;
 - степень 3 — метод Кардано (`kardan`);
 - степень 2 — формулы Виета (`viet`);
 - степень 1 — линейное уравнение;
 - все коэффициенты равны нулю — любое число является корнем.
4. **Извлечение тривиальных корней** (`solve_numerical`). Снимаются замыкающие нулевые коэффициенты, каждый снятый коэффициент добавляет корень $x = 0$.
5. **Оценка радиуса корней** (`findR`). По формуле Коши вычисляется радиус R , ограничивающий область поиска: $R = 1 + \max_i |a_i/a_0|$.
6. **Поиск отрезка изоляции корня** (`findAB`). Сканирование интервала $[-R; R]$ с уменьшающимся шагом до обнаружения смены знака полинома: $f(A) \cdot f(B) < 0$.
7. **Уточнение корня численным методом**:
 - степень ≥ 6 — комбинированный метод хорд и касательных (`khord_method`);
 - степень 5 — метод хорд (`hord_method`);
 - степень 4 — метод касательных (`cas_method`).
8. **Поиск корней чётной кратности** (`_find_even_root`). Если на шаге 7 смена знака не найдена, вычисляются корни производной $f'(x)$; если $f(x_0) \approx 0$ для корня x_0 производной, он принимается как корень чётной кратности.
9. **Понижение степени по схеме Горнера** (`gorn`). Найденный корень исключается делением полинома на $(x - x_0)$, степень уменьшается на

единицу.

10. **Повторение шагов 4–9** до тех пор, пока степень полинома не станет ≤ 3 . При степени 3 применяется метод Кардано, при степени 2 — формулы Виета, при степени 1 — линейное уравнение.
11. **Вывод результатов (main)**. Для каждого найденного корня x_i выводится его значение и значение $f(x_i)$ для проверки.

Входные и выходные данные

В таблице 1 приведено описание всех переменных, используемых в программе.

Table 1 – Описание переменных программы

Переменная	Тип	Границы	Назначение
Глобальные константы			
EPS	float	10^{-9}	Точность вычислений
MAX_ITER	int	1000	Максимальное число итераций
ZERO	float	10^{-15}	Порог для сравнения с нулём
Ввод данных (input_coefficients)			
a	float	$[-10^6; 10^6]$	Коэффициент при x^6
b	float	$[-10^6; 10^6]$	Коэффициент при x^5
c	float	$[-10^6; 10^6]$	Коэффициент при x^4
d	float	$[-10^6; 10^6]$	Коэффициент при x^3
k	float	$[-10^6; 10^6]$	Коэффициент при x^2
m	float	$[-10^6; 10^6]$	Коэффициент при x
n	float	$[-10^6; 10^6]$	Свободный член
Вычисление полинома (p, p_and_dp)			
Mas	list[float]	—	Массив коэффициентов полинома
x	float	$[-R; R]$	Точка вычисления
value	float	$[-10^{15}; 10^{15}]$	Значение полинома
p_val	float	$[-10^{15}; 10^{15}]$	Значение полинома (Горнер)
dp_val	float	$[-10^{15}; 10^{15}]$	Значение производной
N	int	$[0; 6]$	Степень полинома
Схема Горнера (gorn)			
new_Mas	list[float]	—	Коэффициенты частного

Переменная	Тип	Границы	Назначение
Поиск отрезка (findAB, findR)			
A	float	$[-10^4; 10^4]$	Левый конец найденного отрезка
B	float	$[-10^4; 10^4]$	Правый конец найденного отрезка
h	float	$(10^{-7}; 0.1]$	Шаг сканирования
R	float	$[1; 10^4]$	Радиус поиска корней
total	int	$[0; 200000]$	Счётчик итераций сканирования
lead	float	$(0; +\infty)$	$ a_0 $ — модуль старшего коэфф.
max_ratio	float	$[0; +\infty)$	Макс. отношение $ a_i/a_0 $
Метод касательных (cas_method)			
X	list[float]	—	Последовательность приближений
f_val	float	$[-10^{15}; 10^{15}]$	Значение функции
df_val	float	$[-10^{15}; 10^{15}]$	Значение производной
Метод хорд (hord_method)			
x_fix	float	$[A; B]$	Неподвижная точка
x_move	float	$[A; B]$	Подвижная точка
f_fix	float	$[-10^{15}; 10^{15}]$	$f(x_{fix})$
f_move	float	$[-10^{15}; 10^{15}]$	$f(x_{move})$
x_new	float	$[A; B]$	Новое приближение
denom	float	$[-10^{15}; 10^{15}]$	Знаменатель дроби
Комбинированный метод (khord_method)			
x_start	float	$[A; B]$	Точка касательных
x_move	float	$[A; B]$	Точка хорд
f_start	float	$[-10^{15}; 10^{15}]$	$f(x_{start})$
dp_start	float	$[-10^{15}; 10^{15}]$	$f'(x_{start})$
x_start_new	float	$[A; B]$	Новое приближение (касат.)

Переменная	Тип	Границы	Назначение
x_move_new	float	$[A; B]$	Новое приближение (хорд)
Поиск корня чётной кратности (_find_even_root)			
dMas	list[float]	—	Коэффициенты производной
d_degree	int	$[3; 5]$	Степень производной
cands	list[float]	—	Кандидаты (корни $f'(x)$)
Метод Кардано (kardan)			
p_val	float	$[-10^{15}; 10^{15}]$	$p = \frac{3ac-b^2}{3a^2}$
q	float	$[-10^{15}; 10^{15}]$	$q = \frac{2b^3-9abc+27a^2n}{27a^3}$
Delta	float	$[-10^{30}; 10^{30}]$	Дискриминант: $(q/2)^2 + (p/3)^3$
u	float	$[-10^5; 10^5]$	При $\Delta > \varepsilon$: $\sqrt[3]{-q/2 + \sqrt{\Delta}}$; при $\Delta \approx 0$: $\sqrt[3]{-q/2}$
v	float	$[-10^5; 10^5]$	$\sqrt[3]{-q/2 - \sqrt{\Delta}}$; только при $\Delta > \varepsilon$
phi	float	$[0; \pi]$	Аргумент arccos
r	float	$[0; +\infty)$	$2\sqrt{-p/3}$
Формулы Виета (viet)			
D	float	$[-10^{30}; 10^{30}]$	Дискриминант: $b^2 - 4ac$
sqrt_D	float	$[0; +\infty)$	$\sqrt{D}$
Основной решатель (solve_numerical)			
roots	list[float]	—	Найденные корни
step	int	$[1; 6]$	Номер текущего шага
degree	int	$[1; 6]$	Текущая степень полинома
method_func	function	—	Функция выбранного метода

Переменная	Тип	Границы	Назначение
method_name	str	—	Название метода (для вывода)

Примеры работы программы

Ниже приведены результаты тестирования программы для различных степеней уравнения.

Тест 1: уравнение 6-й степени (все корни различны)

Уравнение: $x^6 - 21x^5 + 175x^4 - 735x^3 + 1624x^2 - 1764x + 720 = 0$.

Коэффициенты: $a = 1, b = -21, c = 175, d = -735, k = 1624, m = -1764, n = 720$.

Ожидаемые корни: $x = 1, 2, 3, 4, 5, 6$.

```
>>> khord(a, b, c, d, k, m, n)
Шаг 1: метод комбинированный, x = 1.0000000000
Шаг 2: метод хорд, x = 2.0000000000
Шаг 3: метод касательных, x = 2.9999999999
Шаг 4: kardan, x = [6.0, 4.0000000002, 4.9999999999]
```

Вывод корней:

```
x1 = 1.0000000000; проверка f(x1) = 0.00e+00
x2 = 2.0000000000; проверка f(x2) = 6.34e-10
x3 = 2.9999999999; проверка f(x3) = 1.27e-09
x4 = 6.0000000000; проверка f(x4) = 3.15e-09
x5 = 4.0000000002; проверка f(x5) = 1.91e-09
x6 = 4.9999999999; проверка f(x6) = 2.56e-09
```

Найдены все 6 действительных корней. Проверка подтверждает точность.

Тест 2: уравнение 6-й степени с корнями $x = 0$

Уравнение: $2x^6 + 10x^5 + 12x^4 = 0$, т.е. $2x^4(x + 2)(x + 3) = 0$.

Коэффициенты: $a = 2, b = 10, c = 12, d = 0, k = 0, m = 0, n = 0$.

Ожидаемые корни: $x = 0, 0, 0, 0, -2, -3$.

```
>>> khord(a, b, c, d, k, m, n)
Шаг 1: x = 0 (свободный член = 0)
Шаг 2: x = 0 (свободный член = 0)
Шаг 3: x = 0 (свободный член = 0)
Шаг 4: x = 0 (свободный член = 0)
Шаг 5: viet, x = [-2.0, -3.0]
```

Вывод корней:

```
x1 = 0.0000000000; проверка f(x1) = 0.00e+00
x2 = 0.0000000000; проверка f(x2) = 0.00e+00
x3 = 0.0000000000; проверка f(x3) = 0.00e+00
```

```
x4 = 0.0000000000; проверка f(x4) = 0.00e+00
x5 = -2.0000000000; проверка f(x5) = 0.00e+00
x6 = -3.0000000000; проверка f(x6) = 0.00e+00
```

Все 6 корней найдены: 4 нулевых корня извлечены через снятие замыкающих нулей, оставшиеся 2 — по формулам Виета.

Тест 3: уравнение 5-й степени с кратным корнем

Уравнение: $x^5 - 11x^4 + 45x^3 - 85x^2 + 74x - 24 = 0$, т.е. $(x - 1)^2(x - 2)(x - 3)(x - 4) = 0$.

Коэффициенты: $b = 1, c = -11, d = 45, k = -85, m = 74, n = -24$.

Ожидаемые корни: $x = 1, 1, 2, 3, 4$.

```
>>> hord(b, c, d, k, m, n)
Шаг 1: метод хорд, x = 1.0000000000
Шаг 2: метод касательных, x = 1.0000000000
Шаг 3: kardan, x = [4.0, 2.0, 3.0]

вывод корней:
x1 = 1.0000000000; проверка f(x1) = 7.11e-15
x2 = 1.0000000000; проверка f(x2) = 0.00e+00
x3 = 4.0000000000; проверка f(x3) = 2.84e-13
x4 = 2.0000000000; проверка f(x4) = 2.84e-14
x5 = 3.0000000000; проверка f(x5) = 0.00e+00
```

Найдены все 5 корней, включая двукратный корень $x = 1$.

Тест 4: уравнение 4-й степени

Уравнение: $x^4 - 1 = 0$. Коэффициенты: $c = 1, d = 0, k = 0, m = 0, n = -1$.

Ожидаемые действительные корни: $x = -1, 1$.

```
>>> cas(c, d, k, m, n)
Шаг 1: метод касательных, x = -1.0000000000
Шаг 2: kardan, x = [1.0]

вывод корней:
x1 = -1.0000000000; проверка f(x1) = 0.00e+00
x2 = 1.0000000000; проверка f(x2) = 0.00e+00
```

Найдены 2 действительных корня из 4. Остальные 2 корня — комплексные.

Тест 5: уравнение 3-й степени (Кардано)

Уравнение: $x^3 - 6x^2 + 11x - 6 = 0$. Коэффициенты: $d = 1, k = -6, m = 11, n = -6$.

Ожидаемые корни: $x = 1, 2, 3$.

```
>>> kardan(d, k, m, n)
```

вывод корней:

$x1 = 3.00000000000$; проверка $f(x1) = 0.00e+00$

$x2 = 1.00000000000$; проверка $f(x2) = 0.00e+00$

$x3 = 2.00000000000$; проверка $f(x3) = 0.00e+00$

Все три корня найдены точно.

Тест 6: уравнение 2-й степени (Виета)

Уравнение: $x^2 - 5x + 6 = 0$. Коэффициенты: $k = 1, m = -5, n = 6$.

Ожидаемые корни: $x = 2, 3$.

```
>>> viet(k, m, n)
```

вывод корней:

$x1 = 3.00000000000$; проверка $f(x1) = 0.00e+00$

$x2 = 2.00000000000$; проверка $f(x2) = 0.00e+00$

Оба корня найдены точно.

Тест 7: линейное уравнение

Уравнение: $2x + 4 = 0$. Коэффициенты: $m = 2, n = 4$.

вывод корней:

$x1 = -2.00000000000$; проверка $f(x1) = 0.00e+00$

Корень найден: $x = -2$.

Тест 8: нет решений

Уравнение: $5 = 0$. Коэффициенты: $n = 5$, все остальные равны 0.

Результат: Решений нет

Программа корректно определяет отсутствие решений.

Тест 9: корни чётной кратности

Уравнение: $x^6 - 2x^5 - 17x^4 + 68x^3 - 97x^2 + 62x - 15 = 0$,

т.е. $(x - 1)^4(x + 5)(x - 3) = 0$.

Коэффициенты: $a = 1, b = -2, c = -17, d = 68, k = -97, m = 62, n = -15$.

Ожидаемые корни: $x = 1, 1, 1, 1, -5, 3$.

```
>>> khord(a, b, c, d, k, m, n)
Шаг 1: метод комбинированный, x = -5.0000000000
Шаг 2: метод хорд, x = 3.0000000000
Шаг 3: метод касательных (чётная кратность), x = 1.0000000000
Шаг 4: kardan, x = [1.0, 1.0, 1.0]

вывод корней:
x1 = -5.0000000000; проверка f(x1) = 0.00e+00
x2 = 3.0000000000; проверка f(x2) = 7.67e-12
x3 = 1.0000000000; проверка f(x3) = 7.11e-15
x4 = 1.0000000000; проверка f(x4) = 7.11e-15
x5 = 1.0000000000; проверка f(x5) = 7.11e-15
x6 = 1.0000000000; проверка f(x6) = 7.11e-15
```

Найдены все 6 корней, включая четырёхкратный корень $x = 1$. После нахождения простых корней $x = -5$ и $x = 3$, оставшийся полином $(x - 1)^4$ не меняет знак, поэтому применяется поиск корня чётной кратности через производную.

Блок-схема

Блок-схемы всех подпрограмм алгоритма приведены в Приложении А (стр. 25).

Перечень блок-схем:

1. Главная программа (`All.png`)
2. Поиск отрезка изоляции корня (`otrez.png`)
3. Метод касательных (`casat.png`)
4. Метод хорд (`hord.png`)
5. Комбинированный метод (`combhord.png`)
6. Метод Кардано (`cardan.png`)
7. Формулы Виета (`viet.png`)
8. Схема Горнера (`gorn.png`)

Код программы

Ниже приведён полный исходный код программы на языке Python.

```
1 import math
2
3 EPS = 1e-9
4 MAX_ITER = 1000
5 ZERO = 1e-15
6
7
8 def cbrt(x):
9     return math.copysign(abs(x) ** (1 / 3), x)
10
11
12 # ===== INPUT =====
13
14 def input_coefficients():
15     while True:
16         try:
17             print("ax^6 + bx^5 + cx^4 + dx^3 + kx^2 + mx + n = 0\n")
18             a = float(input("a = "))
19             b = float(input("b = "))
20             c = float(input("c = "))
21             d = float(input("d = "))
22             k = float(input("k = "))
23             m = float(input("m = "))
24             n = float(input("n = "))
25             return a, b, c, d, k, n, m
26         except ValueError:
27             print("\nError!\n")
28
29
30 # ===== p(Mas, x) =====
31
32 def p(Mas, x):
33     value = 0.0
34     for coef in Mas:
35         value = value * x + coef
36     return value
37
38
39 def p_and_dp(Mas, x):
40     N = len(Mas) - 1
41     if N < 0:
42         return 0.0, 0.0
43     p_val = float(Mas[0])
44     dp_val = 0.0
45     for i in range(1, N + 1):
```



```

46         dp_val = dp_val * x + p_val
47         p_val = p_val * x + Mas[i]
48     return p_val, dp_val
49
50
51 def deriv_Mas(Mas):
52     N = len(Mas) - 1
53     if N <= 0:
54         return [0.0]
55     return [Mas[i] * (N - i) for i in range(N)]
56
57
58 def p2(Mas, x):
59     return p(deriv_Mas(deriv_Mas(Mas)), x)
60
61
62 # ===== gorn(Mas, x) =====
63
64 def gorn(Mas, x):
65     if len(Mas) <= 1:
66         return []
67     N = len(Mas)
68     new_Mas = [float(Mas[0])]
69     for i in range(1, N):
70         new_Mas.append(new_Mas[-1] * x + Mas[i])
71     return new_Mas[:-1]
72
73
74 # ===== findAB(Mas) =====
75
76 def findR(Mas):
77     if len(Mas) <= 1 or abs(Mas[0]) < ZERO:
78         return 10.0
79     lead = abs(Mas[0])
80     max_ratio = max(abs(coef) / lead
81                     for coef in Mas[1:]) if len(Mas) > 1 else 0.0
82     return min(1.0 + max_ratio, 1e4)
83
84
85 def findAB(Mas):
86     A, B = 0.0, 0.0
87     h = 0.1
88     R = findR(Mas)
89     total = 0
90     while h > 1e-7:
91         i = -R
92         while i < (R - h):
93             if p(Mas, i) * p(Mas, i + h) <= 0:

```

```

94         A, B = i, i + h
95         return A, B
96     i += h
97     total += 1
98     if total > 200000:
99         return None
100     h /= 10.0
101     return None
102
103
104 # ===== cas_method =====
105
106 def cas_method(Mas, eps=EPS):
107     X = []
108     O = findAB(Mas)
109     if O is None:
110         return None
111     A, B = 0
112     if p(Mas, A) * p2(Mas, A) > 0:
113         X.append(A)
114     elif p(Mas, B) * p2(Mas, B) > 0:
115         X.append(B)
116     else:
117         X.append((A + B) / 2)
118     i = 0
119     while i < MAX_ITER:
120         f_val, df_val = p_and_dp(Mas, X[i])
121         if abs(df_val) < ZERO:
122             return None
123         X.append(X[i] - f_val / df_val)
124         if abs(X[i + 1] - X[i]) < eps:
125             return X[i + 1]
126         i += 1
127     return None
128
129
130 def cas(a, b, c, d, k, m, n, eps=EPS):
131     Mas = [a, b, c, d, k, m, n]
132     return cas_method(Mas, eps)
133
134
135 # ===== hord_method =====
136
137 def hord_method(Mas, eps=EPS):
138     X = []
139     O = findAB(Mas)
140     if O is None:
141         return None

```

```

142     A, B = 0
143     x_fix = B
144     x_move = A
145     f_fix = p(Mas, x_fix)
146     f_move = p(Mas, x_move)
147     n = 0
148     while n < MAX_ITER:
149         if abs(f_move) < eps:
150             X.append(x_move)
151             return x_move
152         if abs(f_fix) < eps:
153             x_move = x_fix
154             X.append(x_move)
155             return x_move
156         denom = f_fix - f_move
157         if abs(denom) < ZERO:
158             return None
159         x_new = x_move - f_move * (x_fix - x_move) / denom
160         f_new = p(Mas, x_new)
161         if abs(x_new - x_move) < eps:
162             X.append(x_new)
163             return x_new
164         x_move = x_new
165         f_move = f_new
166         n += 1
167     return None
168
169
170 def hord(b, c, d, k, m, n, eps=EPS):
171     Mas = [b, c, d, k, m, n]
172     return hord_method(Mas, eps)
173
174
175 # ===== khord_method =====
176
177 def khord_method(Mas, eps=EPS):
178     X = []
179     O = findAB(Mas)
180     if O is None:
181         return None
182     A, B = 0
183     if p(Mas, A) * p2(Mas, A) > 0:
184         x_start = A
185         x_move = B
186     elif p(Mas, B) * p2(Mas, B) > 0:
187         x_start = B
188         x_move = A
189     else:

```

```

190         x_start = (A + B) / 2
191         x_move = A
192         f_start = p(Mas, x_start)
193         f_move = p(Mas, x_move)
194         n = 0
195         while True:
196             _, dp_start = p_and_dp(Mas, x_start)
197             if abs(dp_start) < ZERO:
198                 return None
199             x_start_new = x_start - f_start / dp_start
200             f_start_new = p(Mas, x_start_new)
201             denom = f_start_new - f_move
202             if abs(denom) < ZERO:
203                 return x_start_new if abs(f_start_new) <= abs(f_move) \
204                     else x_move
205             x_move_new = x_move - f_move * (x_start_new - x_move) / denom
206             f_move_new = p(Mas, x_move_new)
207             X.append((x_start_new + x_move_new) / 2)
208             if abs(x_start_new - x_move_new) < eps:
209                 return X[n]
210             x_start = x_start_new
211             x_move = x_move_new
212             f_start = f_start_new
213             f_move = f_move_new
214             n += 1
215             if not (n < MAX_ITER):
216                 return None
217
218
219 def khord(c, d, k, m, n, eps=EPS):
220     Mas = [c, d, k, m, n]
221     return khord_method(Mas, eps)
222
223
224 # ===== _find_even_root =====
225
226 def _find_even_root(Mas, eps=EPS):
227     dMas = deriv_Mas(Mas)
228     if len(dMas) <= 1:
229         return None
230     d_degree = len(dMas) - 1
231     if d_degree == 1:
232         cand = [-dMas[1] / dMas[0]]
233     elif d_degree == 2:
234         cand = viet(dMas[0], dMas[1], dMas[2])
235     elif d_degree == 3:
236         cand = kardan(dMas[0], dMas[1], dMas[2], dMas[3])
237     else:

```

```

238     r = findAB(dMas)
239     if r is None:
240         return None
241     A, B = r
242     x0 = (A + B) / 2
243     for _ in range(MAX_ITER):
244         fv, dfv = p_and_dp(dMas, x0)
245         if abs(dfv) < ZERO:
246             break
247         x1 = x0 - fv / dfv
248         if abs(x1 - x0) < eps:
249             x0 = x1
250             break
251         x0 = x1
252     cands = [x0]
253     for c in cands:
254         if abs(p(Mas, c)) < 1e-6:
255             return c
256     return None
257
258
259 # ===== solve_numerical =====
260
261 def _method_for_degree(degree):
262     if degree >= 6:
263         return khord_method, "combined"
264     if degree == 5:
265         return hord_method, "chord"
266     return cas_method, "tangent" # degree == 4
267
268
269 def solve_numerical(Mas):
270     roots = []
271     Mas = [float(coef) for coef in Mas]
272     step = 1
273     while len(Mas) > 1:
274         while len(Mas) > 1 and abs(Mas[0]) < ZERO:
275             Mas = Mas[1:]
276         if len(Mas) <= 1:
277             break
278         while len(Mas) > 1 and abs(Mas[-1]) < ZERO:
279             roots.append(0.0)
280             Mas = Mas[:-1]
281             step += 1
282         if len(Mas) <= 1:
283             break
284         degree = len(Mas) - 1
285         if degree == 1:

```

```

286         x = -Mas[1] / Mas[0]
287         roots.append(x)
288         break
289     if degree == 2:
290         new_roots = viet(Mas[0], Mas[1], Mas[2])
291         roots.extend(new_roots)
292         break
293     if degree == 3:
294         new_roots = kardan(Mas[0], Mas[1], Mas[2], Mas[3])
295         roots.extend(new_roots)
296         break
297     method_func, method_name = _method_for_degree(degree)
298     x = method_func(Mas)
299     if x is None:
300         x = _find_even_root(Mas)
301         if x is None:
302             break
303     roots.append(x)
304     Mas = gorn(Mas, x)
305     step += 1
306     return roots
307
308
309 # ===== kardan(d, k, m, n) =====
310
311 def kardan(d, k, m, n):
312     a = d
313     b = k
314     c = m
315     free = n
316     p_val = (3 * a * c - b * b) / (3 * a * a)
317     q = (2 * b ** 3 - 9 * a * b * c + 27 * a * a * free) \
318         / (27 * a ** 3)
319     Delta = (q / 2) ** 2 + (p_val / 3) ** 3
320     if Delta > EPS:
321         u = cbrt(-q / 2 + math.sqrt(Delta))
322         v = cbrt(-q / 2 - math.sqrt(Delta))
323         x = u + v - b / (3 * a)
324         return [x]
325     if abs(Delta) <= EPS:
326         if abs(q) <= EPS:
327             x = -b / (3 * a)
328             return [x, x, x]
329         u = cbrt(-q / 2)
330         x1 = 2 * u - b / (3 * a)
331         x2 = -u - b / (3 * a)
332         return [x1, x2, x2]
333     phi = math.acos(-q / (2 * math.sqrt(-(p_val / 3) ** 3)))

```

```

334     r = 2 * math.sqrt(-p_val / 3)
335     x1 = r * math.cos(phi / 3) - b / (3 * a)
336     x2 = r * math.cos((phi + 2 * math.pi) / 3) - b / (3 * a)
337     x3 = r * math.cos((phi + 4 * math.pi) / 3) - b / (3 * a)
338     return [x1, x2, x3]
339
340
341 # ===== viet(k, m, n) =====
342
343 def viet(k, m, n):
344     a = k
345     b = m
346     c = n
347     D = b * b - 4 * a * c
348     if D < -EPS:
349         return []
350     if abs(D) <= EPS:
351         x = -b / (2 * a)
352         return [x, x]
353     sqrt_D = math.sqrt(D)
354     x1 = (-b + sqrt_D) / (2 * a)
355     x2 = (-b - sqrt_D) / (2 * a)
356     return [x1, x2]
357
358
359 # ===== main() =====
360
361 def main():
362     a, b, c, d, k, n, m = input_coefficients()
363     roots = None
364     if a != 0:
365         Mas = [a, b, c, d, k, m, n]
366         roots = solve_numerical(Mas)
367     elif b != 0:
368         Mas = [b, c, d, k, m, n]
369         roots = solve_numerical(Mas)
370     elif c != 0:
371         Mas = [c, d, k, m, n]
372         roots = solve_numerical(Mas)
373     elif d != 0:
374         roots = kardan(d, k, m, n)
375     elif k != 0:
376         roots = viet(k, m, n)
377     elif m != 0:
378         x = -(n / m)
379         roots = [x]
380     elif n != 0:
381         roots = []

```

```

382     else:
383         roots = None
384     if roots is None:
385         print("Any number is a root")
386         return
387     if len(roots) == 0:
388         print("No solutions")
389         return
390     for i, x in enumerate(roots, 1):
391         f_x = a*x**6 + b*x**5 + c*x**4 + d*x**3 \
392             + k*x**2 + m*x + n
393         print(f"  x{i} = {x:.10f}; f(x{i}) = {f_x:.2e}")
394
395
396 if __name__ == "__main__":
397     main()

```


Заключение

В ходе выполнения лабораторной работы был реализован алгоритм решения уравнения шестой степени с использованием различных численных методов: комбинированного метода хорд и касательных, метода хорд, метода касательных (Ньютона), метода Кардано и формул Виета.

Программа автоматически определяет степень уравнения по старшему ненулевому коэффициенту и применяет соответствующий метод. После нахождения корня степень понижается с помощью схемы Горнера, и для оставшегося полинома выбирается метод, соответствующий новой степени.

Реализованы дополнительные механизмы для повышения надёжности:

- извлечение тривиальных корней $x = 0$ путём снятия замыкающих нулевых коэффициентов;
- поиск корней чётной кратности через анализ производной — если корень $f'(x) = 0$ одновременно обращает в ноль $f(x)$, он является корнем чётной кратности;
- ограничение итераций при сканировании отрезка для предотвращения зависания при отсутствии действительных корней;
- возврат кратных корней в методах Кардано и Виета с учётом кратности.

Тестирование подтвердило корректность работы программы для всех девяти тестовых случаев: уравнения 6-й степени (с различными и кратными корнями, включая $x = 0$), уравнения 5-й степени, 4-й, 3-й, 2-й, линейного уравнения, а также случая отсутствия решений. Все найденные корни подтверждены подстановкой в исходное уравнение.

Приложение А. Блок-схемы алгоритма

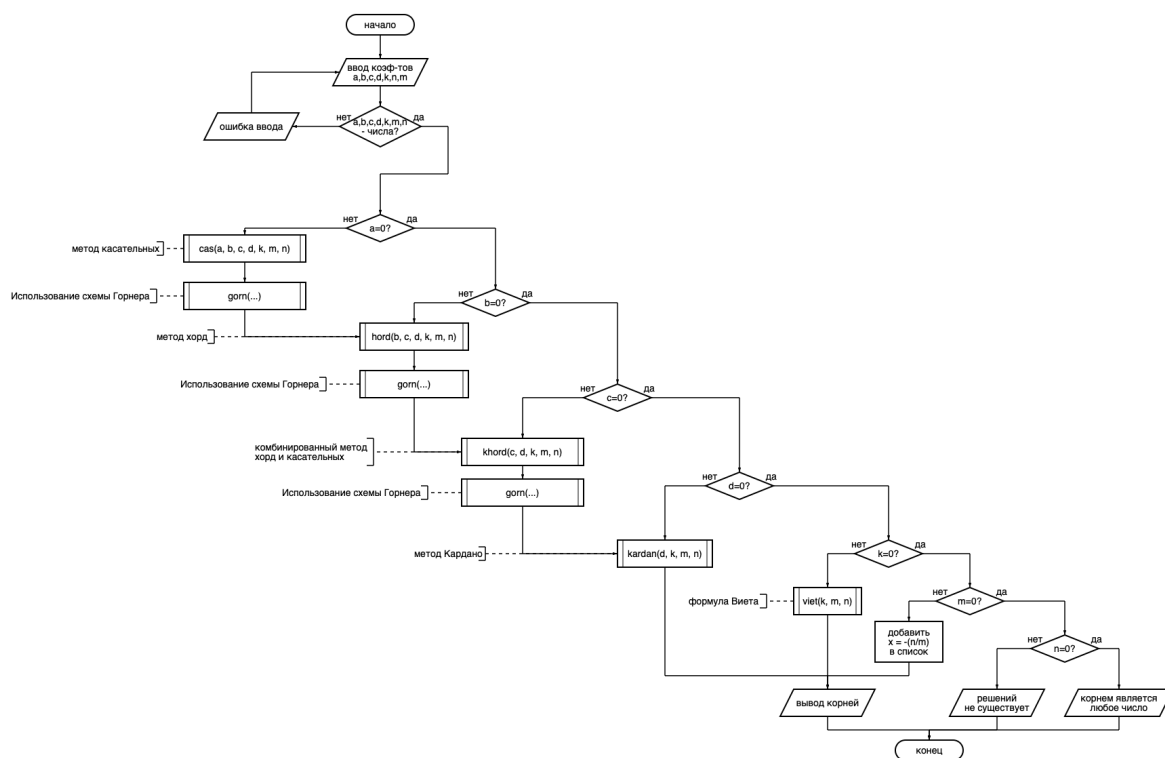


Figure 1 – Главная программа

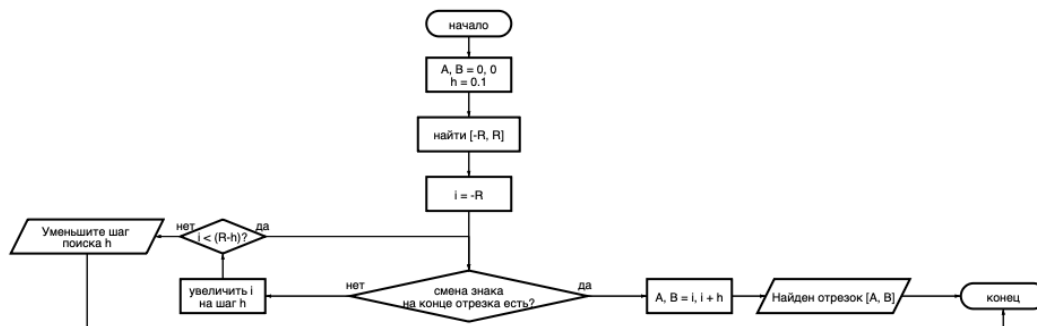


Figure 2 – Поиск отрезка изоляции корня (findAB)

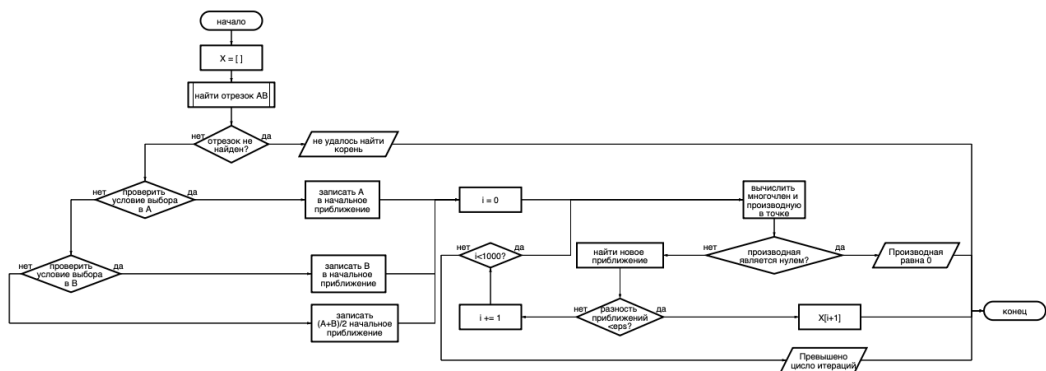


Figure 3 – Метод касательных (cas_method)

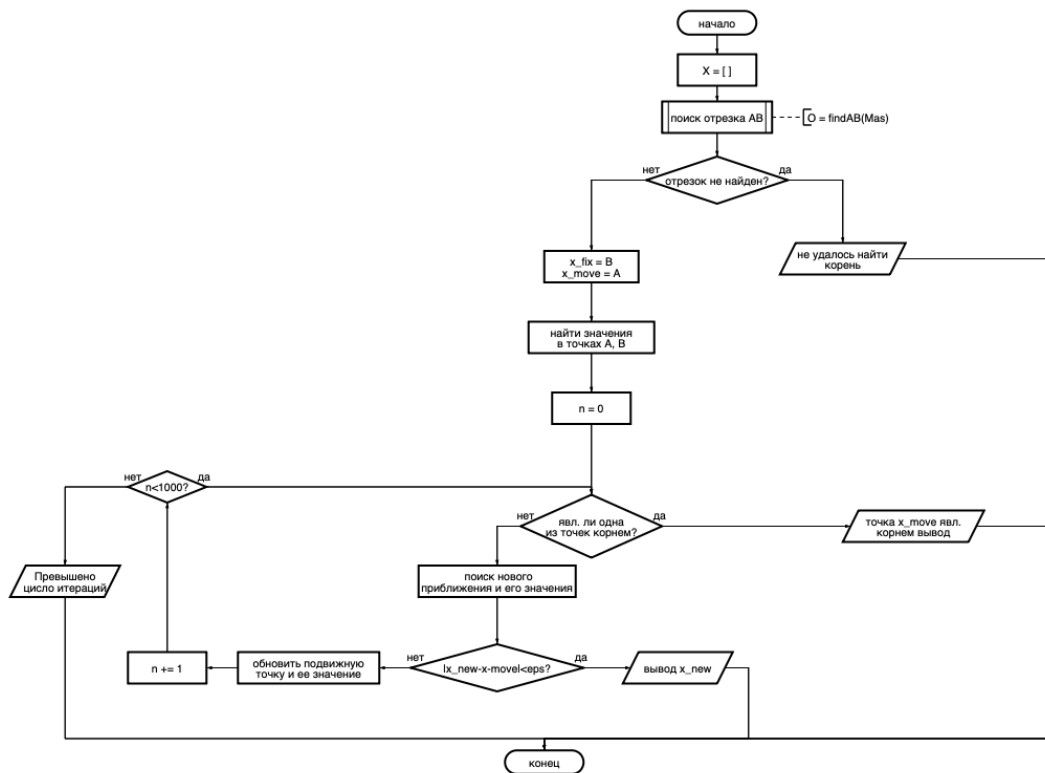


Figure 4 – Метод хорд (hord_method)

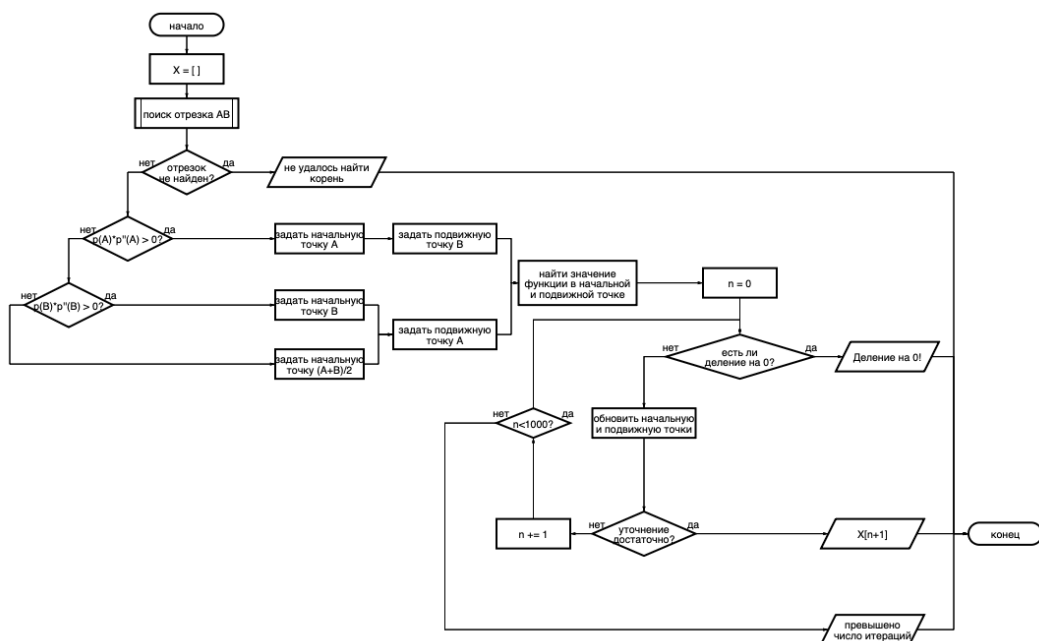


Figure 5 – Комбинированный метод (khord_method)

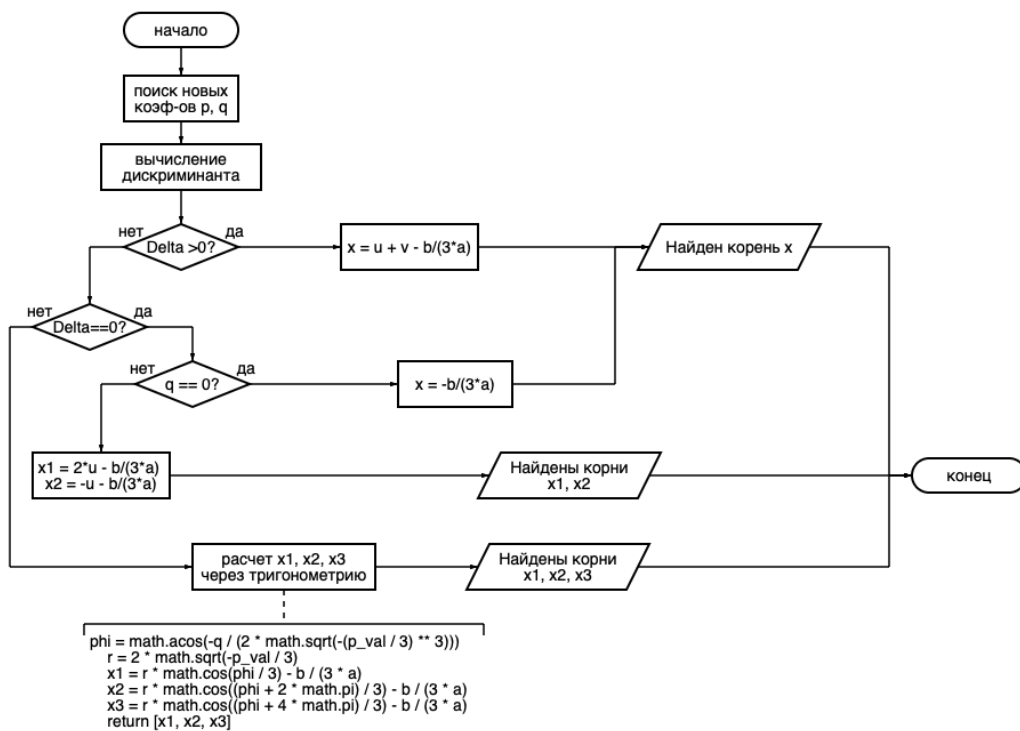


Figure 6 – Метод Кардано (kardan)

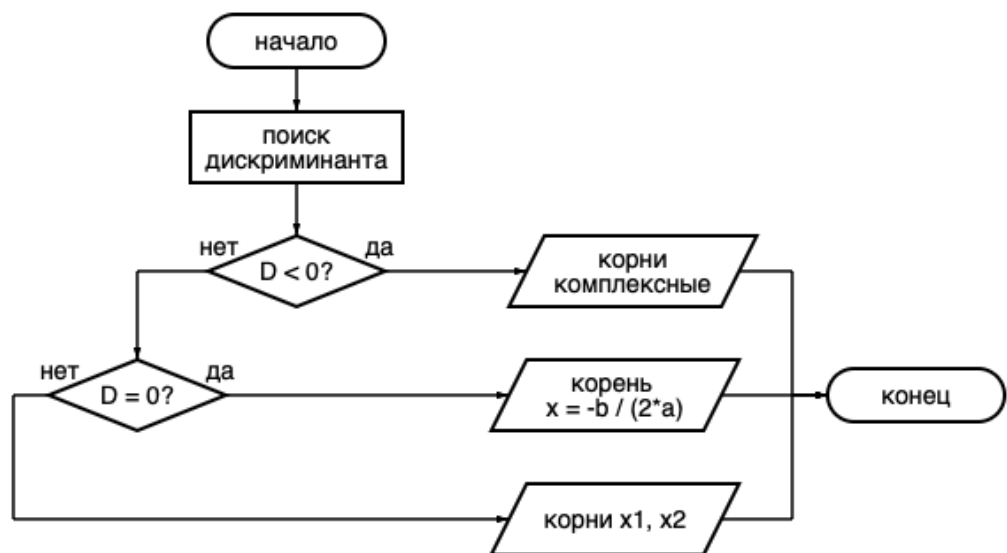


Figure 7 – Формулы Виета (viet)

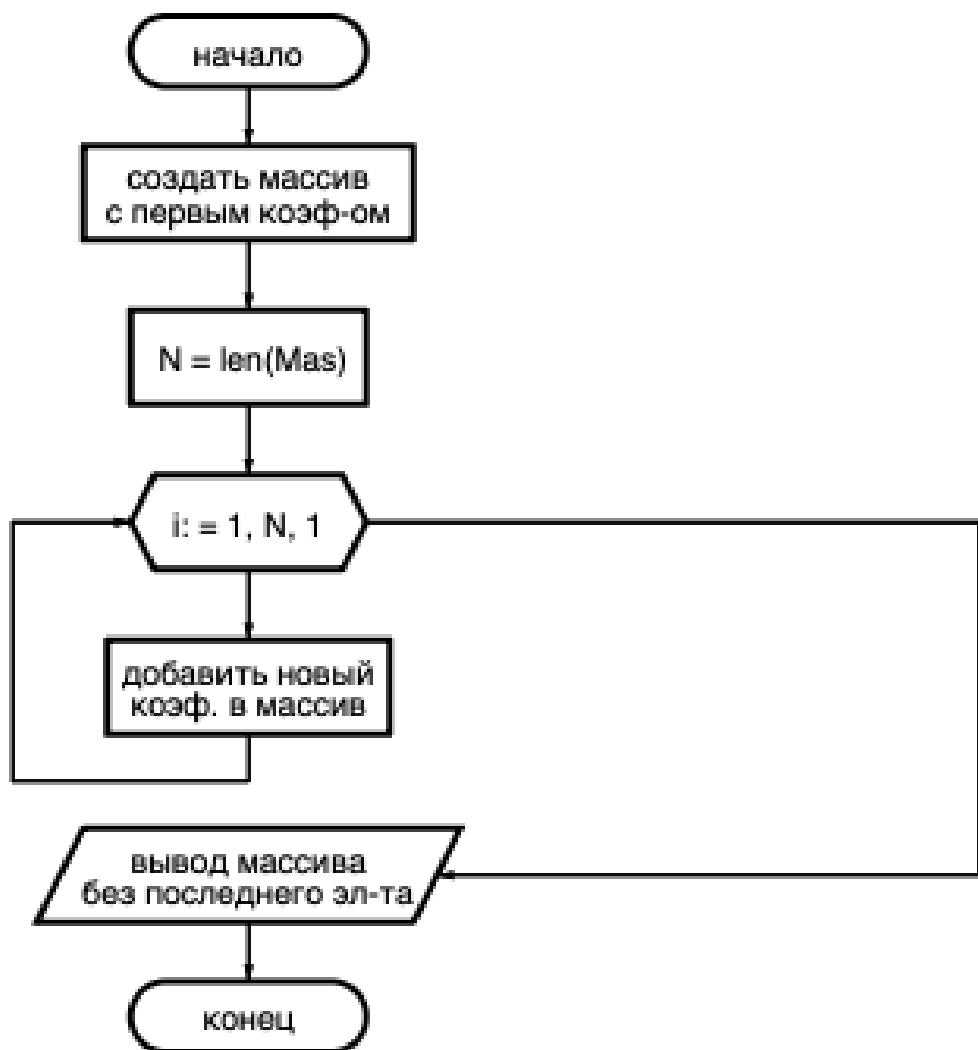


Figure 8 – Схема Горнера (gorn)